ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

NHẬP MÔN TRÍ TUỆ NHÂN TẠO - CO3061

BÁO CÁO BÀI TẬP LỚN

Lớp: L03

GVHD:	Th.S Vương Bá Thịnh	
SV thực hiện:	Đặng Đình Minh Hoàng	2411068
	Phan Ngọc Xuân Lợi	2411969
	Phạm Phùng Đăng Khoa	2433156
	Nguyễn Ngọc Trường Khang	2411455

TP Hồ Chí Minh, Tháng 3 Năm 2026

Mục lục

Danh sách thành viên và công việc **2**

Lời mở đầu **3**

1 Giới thiệu **4**

2 Trò chơi Sudoku **5**
2.1 Mô tả trò chơi . . . 5
2.1.1 Luật chơi . . . 5
2.1.2 Biểu diễn bài toán và không gian trạng thái . . . 6
2.2 Giải thuật Blind Search . . . 7
2.2.1 Mô tả giải thuật sử dụng . . . 7
2.2.2 Mô hình hóa bài toán và hiên thực . . . 7
2.2.3 Kết quả chạy thử . . . 7
2.2.4 Phân tích hiệu suất . . . 8
2.3 Giải thuật Heuristic trong việc giải bài toán Sudoku . . . 10
2.3.1 Giới thiệu . . . 10
2.3.2 Môt vài công trình tương tự . . . 10
2.3.3 Hiện thực giải pháp . . . 11
2.3.4 Kết quả . . . 14
2.4 So sánh giữa Blind Search (DFS) và Heuristic (MRV + LCV) . . . 15

3 Trò chơi Chess Ranger **17**
3.1 Mô tả trò chơi . . . 17
3.1.1 Luật chơi . . . 17
3.1.2 Biểu diễn bài toán và không gian trạng thái . . . 18
3.2 Giải thuật Blind Search . . . 19
3.2.1 Mô tả giải thuật sử dụng . . . 19
3.2.2 Mô hình hóa bài toán và hiên thực . . . 19
3.2.3 Kết quả chạy thử . . . 19
3.2.4 Phân tích hiệu suất của DFS . . . 20
3.3 Giải thuật Heuristic . . . 21
3.3.1 Mô tả giải thuật sử dụng . . . 21
3.3.2 Mô hình hóa bài toán và hiên thực . . . 21
3.3.3 Kết quả thực hiện . . . 23
3.3.4 Phân tích hiệu suất . . . 23
3.4 So sánh DFS và Heuristic (A*) . . . 24
3.4.1 Bảng kết quả thực nghiệm . . . 24
3.4.2 Nhận xét kết quả . . . 24

Phụ lục **27**

Danh sách thành viên và công việc

Họ tên	**MSSV**	**Công việc**	**Hoàn thành**	**Nhận xét**
Đặng Đình Minh Hoàng	2411068	Sudoku Heuristic	100%	
Phan Ngọc Xuân Lợi	2411969	Sudoku Blind Search	100%	
Phạm Phùng Đăng Khoa	2433156	Chess Ranger Heuristic	100%	
Nguyễn Ngọc Trường Khang	2411455	Chess Ranger Blind Search	100%	

Lời mở đầu

Chúng em xin trân trọng gửi lời cảm ơn sâu sắc đến quý thầy, cô Trường Đại học Bách Khoa – Đại học Quốc gia TP. Hồ Chí Minh. Chính nhờ sự tận tâm trong giảng dạy, sự truyền đạt đầy tâm huyết những kiến thức quý báu, cùng môi trường học tập thuận lợi mà thầy cô đã tạo dựng, chúng em đã có cơ hội học hỏi, nghiên cứu và phát triển bản thân một cách hiệu quả trong suốt quá trình học tập.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn chân thành đến giảng viên hướng dẫn: **Thầy Vương Bá Thịnh**. Thầy đã dành nhiều thời gian theo sát quá trình thực hiện bài tập lớn của chúng em, tận tình hướng dẫn, đưa ra những nhận xét sâu sắc và góp ý quý báu. Sự chỉ dẫn chu đáo và khắt khe đầy tâm huyết của thầy không chỉ giúp chúng em hoàn thiện bài tập lớn, mà còn góp phần rèn luyện kỹ năng tư duy logic, trình bày khoa học và tiếp cận vấn đề một cách hiệu quả.

Quá trình thực hiện bài tập lớn này là cơ hội quý giá để chúng em củng cố kiến thức lý thuyết, tiếp cận thực tế và mở rộng tầm nhìn chuyên môn. Dù phải đối mặt với không ít khó khăn và thử thách, nhưng đây thật sự là một trải nghiệm học tập bổ ích, giúp chúng em tích lũy thêm hành trang vững chắc cho kỳ thi sắp tới và chặng đường học thuật phía trước.

Cuối cùng, chúng em xin kính chúc thầy Thịnh luôn dồi dào sức khỏe, tràn đầy nhiệt huyết và tiếp tục là nguồn động lực, kim chỉ nam vững chắc giúp sinh viên tự tin trên con đường chinh phục tri thức và xây dựng tương lai.

1 Giới thiệu

Trong dòng chảy phát triển của Trí tuệ Nhân tạo (Artificial Intelligence - AI), các giải thuật tìm kiếm luôn giữ vị trí hạt nhân, đóng vai trò là cơ chế thực thi cốt lõi cho các hệ thống ra quyết định và tối ưu hóa tổ hợp. Về bản chất, hầu hết các bài toán thực tế phức tạp—từ lập lịch sản xuất, định tuyến logistics cho đến các hệ thống tự hành—đều có thể được chuyển đổi thành bài toán tìm kiếm trên không gian trạng thái (state space representation). Mục tiêu của quá trình này là xác định một quỹ đạo tối ưu các hành động $(a_1, a_2, ..., a_n)$ nhằm chuyển đổi hệ thống từ trạng thái khởi đầu (S_{start}) đến trạng thái mục tiêu (S_{goal}) mà vẫn đảm bảo thỏa mãn tập hợp các ràng buộc khắt khe về tài nguyên và thời gian.

Tuy nhiên, thách thức lớn nhất trong lĩnh vực này chính là hiện tượng "bùng nổ tổ hợp" (combinatorial explosion). Khi kích thước của không gian trạng thái tăng theo hàm mũ so với quy mô bài toán, các phương pháp tiếp cận truyền thống thường gặp khó khăn về mặt hiệu năng. Các phương pháp tìm kiếm trong AI hiện nay được phân chia thành hai chiến lược tiếp cận chủ đạo:

1. Tìm kiếm không có thông tin (Uninformed Search): Các giải thuật như Tìm kiếm theo chiều sâu (DFS) hay Tìm kiếm theo chiều rộng (BFS) vận hành dựa trên cơ chế duyệt vét cạn mà không có bất kỳ tri thức tiên nghiệm nào về cấu trúc bài toán. Mặc dù đảm bảo tính đầy đủ (completeness) trong một số điều kiện nhất định, các phương pháp này thường tiêu tốn tài nguyên tính toán cực lớn. Đối với các bài toán có độ sâu mục tiêu lớn, việc phải duyệt qua hàng triệu trạng thái không liên quan khiến chúng trở nên bất khả thi trong các ứng dụng thời gian thực.

2. Tìm kiếm có tri thức (Informed Search/Heuristic Search): Đây là hướng tiếp cận tiên tiến hơn, sử dụng các hàm đánh giá $h(n)$ để ước lượng chi phí còn lại từ trạng thái hiện tại đến đích. Bằng cách tích hợp tri thức chuyên gia hoặc các đặc trưng của bài toán vào hàm heuristic, thuật toán có thể ưu tiên thăm dò các nhánh có xác suất dẫn đến lời giải cao nhất.

- Các chiến thuật như *Minimum Remaining Values* (MRV) và *Least Constraining Value* (LCV) đóng vai trò sống còn trong việc giải quyết các bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problems - CSP).
- Thuật toán A^* vẫn là một tiêu chuẩn vàng khi kết hợp giữa chi phí thực tế đã đi qua và chi phí ước lượng, đảm bảo tìm ra lời giải tối ưu nếu hàm heuristic thỏa mãn điều kiện chấp nhận được (admissible).

Đặc biệt, trong các bài toán có không gian tìm kiếm đặc biệt lớn và phức tạp như Sudoku, giải cờ, hay lập kế hoạch cho robot, việc thiết kế một hàm Heuristic hiệu quả không chỉ đơn thuần là kỹ thuật bổ trợ mà đã trở thành yếu tố quyết định đến khả năng hội tụ và tính khả thi của giải thuật trên các hệ thống tính toán hiện đại.

2 Trò chơi Sudoku

2.1 Mô tả trò chơi

2.1.1 Luật chơi

Trò chơi được ham khảo trên trang web **Sudoku**, mọi người có thể truy cập vào để tìm hiểu rõ hơn và chơi thử nếu muốn.

Sudoku là một trò chơi logic puzzle rất phổ biến trên thế giới, yêu cầu người chơi điền các con số vào một bảng sao cho thỏa mãn các quy tắc nhất định. Trò chơi này không đòi hỏi khả năng tính toán phức tạp mà chủ yếu dựa vào tư duy logic và khả năng suy luận.

Một bảng Sudoku tiêu chuẩn có kích thước 9x9. Bảng này được chia thành 81 ô vuông nhỏ và được phân thành 9 khối (subgrid), mỗi khối có kích thước: 3x3

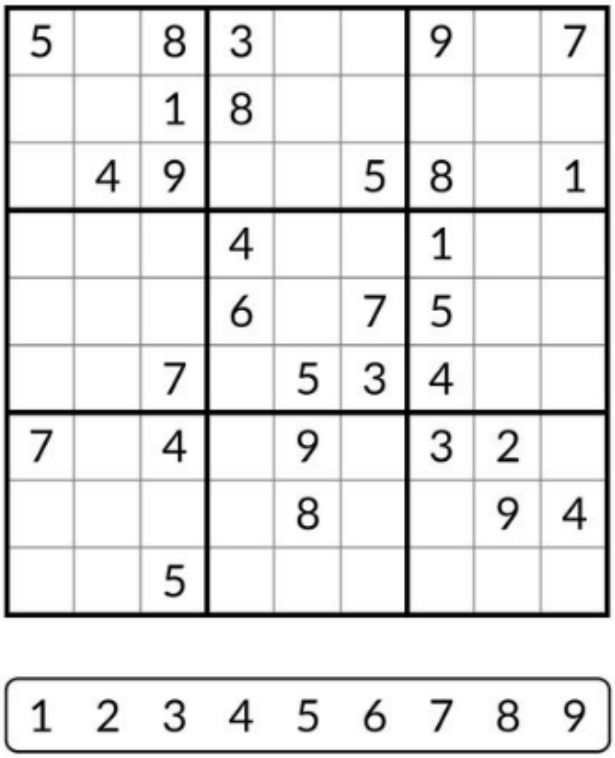

Figure 1: *Một bảng Sudoku ban đầu*

Mục tiêu của trò chơi là điền các số từ 1 đến 9 vào các ô trống sao cho thỏa mãn ba quy tắc sau:

Quy tắc hàng: Mỗi hàng của bảng Sudoku phải chứa tất cả các số từ 1 đến 9, và không được lặp lại số nào.

Quy tắc cột: Mỗi cột của bảng Sudoku cũng phải chứa các số từ 1 đến 9, và không được trùng lặp.

Quy tắc khối: Mỗi khối 3×3 trong bảng Sudoku phải chứa các số từ 1 đến 9, mỗi số chỉ xuất hiện một lần duy nhất.

2.1.2 Biểu diễn bài toán và không gian trạng thái

Trạng thái bắt đầu (S_0): Một đề bài Sudoku sẽ được biểu diễn bằng một ma trận 2 chiều 9×9. Các ô trên ma trận này sẽ có sẵn các số nguyên giá trị từ 1 đến 9 ngẫu nhiên (do đề bài cung cấp). Các ô trống (ta sẽ tiến hành điền các số vào ô trống này để tìm lời giải) sẽ được gán giá trị theo quy ước nhất định (thường là 0 hoặc "."). Đây sẽ chính là nút gốc (root) của cây tìm kiếm không gian trạng thái.

Trạng thái kết thúc (Goal State): Trò chơi sẽ kết thúc khi toàn bộ 81 ô trên ma trận đều được lấp đầy, có nghĩa không còn ô nào mang giá trị là 0 (hoặc "." tùy theo quy ước của bài toán). Đồng thời, sự phân bố các giá trị nguyên trên ma trận đó phải thỏa mãn hoàn toàn các ràng buộc của trò chơi: mỗi hàng, mỗi cột và mỗi khối con 3×3 đều chứa các tập chữ số từ 1 đến 9 mà không có sự trùng lặp.

Luật di chuyển: Tại một trạng thái S bất kỳ đang xét trên cây tìm kiếm, ta sẽ xác định một ô trống tiếp theo cần điền. Thử gán một giá trị $v \in \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ vào ô trống đó. Điều kiện để hành động này hợp lệ là giá trị v chưa từng xuất hiện trong cùng hàng, cùng cột, và cùng khối 3×3 chứa ô trống đang xét. Khi việc gán hợp lệ, cây tìm kiếm sẽ sinh ra một trạng thái con mới để tiếp tục tìm ra lời giải bài toán.

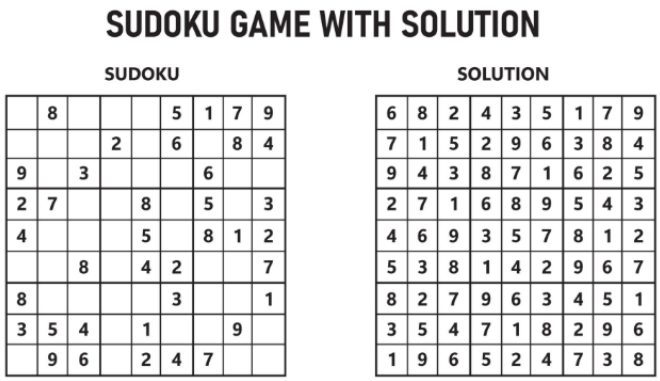

Figure 2: *Đề bài Sudoku và lời giải của nó.*

2.2 Giải thuật Blind Search

2.2.1 Mô tả giải thuật sử dụng

Giải thuật Tìm kiếm theo chiều sâu (Depth-First Search - DFS) thuộc nhóm giải thuật tìm kiếm mù (Blind Search), nghĩa là quá trình duyệt không gian trạng thái diễn ra một cách hệ thống theo một trình tự cố định mà không sử dụng bất kỳ hàm đánh giá hay thông tin gợi ý nào để ưu tiên các nhánh.

Trong bài toán Sudoku, DFS hoạt động dựa trên cơ chế thử và sai (Trial and Error) kết hợp quay lui (Backtracking). Thuật toán tiến hành duyệt không gian trạng thái theo chiều sâu bằng cách liên tục điền các giá trị hợp lệ (từ 1 đến 9) vào các ô trống. Khi một giá trị được chấp nhận, giải thuật sinh ra một trạng thái mới và tiếp tục tiến sâu hơn vào ô trống tiếp theo để xây dựng lời giải. Quá trình này tiếp diễn cho đến khi toàn bộ bảng được lấp đầy.

Tuy nhiên, trong quá trình duyệt, thuật toán có thể chạm đến trạng thái "ngõ cụt" - trạng thái mà tại một ô trống, không có bất kỳ giá trị nào từ 1 đến 9 thỏa mãn các ràng buộc của trò chơi. Lúc này, cơ chế quay lui được kích hoạt tuân theo nguyên tắc LIFO (Last In, First Out) tương tự như cấu trúc dữ liệu Stack: thuật toán sẽ lùi lại trạng thái của ô trống ngay trước đó, xóa bỏ giá trị hiện tại và thử nghiệm nhánh giá trị ứng viên tiếp theo.

2.2.2 Mô hình hóa bài toán và hiên thực

Để áp dụng DFS, trò chơi Sudoku được mô hình hóa thành một bài toán tìm kiếm không gian trạng thái với các thành phần sau: Trạng thái khởi đầu, trạng thái kết thúc, Chiến lược duyệt và Luật chuyển trạng thái.

Thứ nhất, trạng thái khởi đầu được khởi tạo ở dạng bảng 9x9 ban đầu do đề bài cung cấp, trong đó một số ô đã chứa sẵn giá trị hợp lệ và các ô còn lại ở trạng thái trống.

Thứ hai, trạng thái kết thúc là trạng thái mà toàn bộ 81 ô trên bảng lưới đều đã được điền số. Đồng thời, trạng thái này phải thỏa mãn tuyệt đối các ràng buộc của trò chơi: không có sự trùng lặp chữ số (từ 1 đến 9) trên bất kỳ hàng ngang, cột dọc hay phân vùng 3x3 nào.

Thứ ba, chiến lược duyệt được tiến hành một cách tuần tự theo hướng từ trái sang phải, từ trên xuống dưới (bắt đầu từ ô trên cùng bên trái). Tại mỗi ô trống, tập giá trị ứng viên được duyệt cố định theo thứ tự tăng dần từ 1 đến 9.

Thứ tư, tại một trạng thái hiện tại đang xét một ô trống, thuật toán thử lần lượt các giá trị ứng viên. Nếu một giá trị được điền không vi phạm luật Sudoku (chưa xuất hiện ở hàng, cột và khối 3x3 tương ứng), một hành động hợp lệ được ghi nhận và một trạng thái con mới được sinh ra. Nếu tất cả các giá trị từ 1 đến 9 đều vi phạm ràng buộc, hệ thống xác định đây là một nhánh cụt, từ đó thực hiện hành động quay lui về trạng thái cha liền trước để tiếp tục khám phá.

2.2.3 Kết quả chạy thử

Quá trình thực nghiệm được tiến hành bằng cách dùng thuật toán DFS để giải các đề bài Sudoku với mức độ khó khác nhau. Độ khó được phân loại dựa trên các đề bài trên trang web (Puzzle Sudoku). Mục đích của việc thực nghiệm này là thu thập số liệu thực tế về thời gian tìm ra lời giải, sự tiêu tốn bộ nhớ và số lượng trạng thái đã duyệt.

Tất cả các đề bài trong quá trình thử nghiệm đều có một lời giải duy nhất. Các trạng thái kết thúc (trạng thái đích) phải là các trạng thái hợp lệ, lấp đầy 81 ô và không vi phạm các ràng buộc của trò chơi. Dưới đây là bảng số liệu thu thập được trong quá trình thực nghiệm:

Table 1: Bảng thống kê hiệu suất giải thuật DFS đối với các Input

Cấu hình Input	Giải thuật	Thời gian thực thi (giây)	Tiêu tốn bộ nhớ (MB)	Số nút đã duyệt	Kết quả
EASY_1	DFS	0.04451	0.00363	10472	Thành công
EASY_2	DFS	0.15746	0.00363	43347	Thành công
INTERMEDIATE_1	DFS	0.04037	0.00310	10775	Thành công
INTERMEDIATE_2	DFS	1.05500	0.00354	272870	Thành công
ADVANCED_1	DFS	0.20496	0.00328	54708	Thành công
ADVANCED_2	DFS	0.05516	0.00337	10073	Thành công
EVIL_1	DFS	0.51901	0.00354	143152	Thành công
EVIL_2	DFS	0.26863	0.00346	75199	Thành công

2.2.4 Phân tích hiệu suất

Dựa trên bảng số liệu thực nghiệm, hiệu suất của giải thuật DFS khi áp dụng vào bài toán Sudoku được phân tích qua hai khía cạnh cốt lõi: độ phức tạp không gian (sự tiêu tốn bộ nhớ) và độ phức tạp thời gian (thời gian thực thi và số nút đã duyệt).

Độ phức tạp không gian

Bảng số liệu cho thấy khi sử dụng DFS, dung lượng bộ nhớ tiêu thụ thấp và gần như ổn định ở mức 0.003 MB, hoàn toàn không phụ thuộc vào độ khó của đề bài hay số lượng trạng thái đã duyệt qua. Do DFS chỉ cần lưu lại đường đi hiện tại từ nút gốc đến trạng thái đang xét trong quá trình đệ quy, nên lượng bộ nhớ sử dụng khá nhỏ. Khi thuật toán đi sai hướng, quá trình backtracking sẽ xảy ra và các trạng thái thuộc nhánh đó sẽ không còn được lưu trữ nữa, vì vậy vùng nhớ tương ứng cũng được giải phóng.

Độ phức tạp thời gian

Đáng chú ý nhất là sự bất tương xứng rõ rệt giữa mức độ khó của đề bài và chi phí thời gian/số nút duyệt của giải thuật. Cụ thể, đề bài INTERMEDIATE_2 tốn nhiều thời gian nhất (1.055 giây) và phải duyệt một lượng trạng thái lớn ($272,870$ nút), lớn hơn số trạng thái được duyệt ở mức độ khó hơn như ADVANCED_2 ($10,073$ nút) hay thậm chí là EVIL_1 ($143,152$ nút). Điều này xuất phát từ sự khác biệt giữa logic giải quyết vấn đề của con người và cơ chế hoạt động của thuật toán tìm kiếm mù.

Theo góc nhìn của con người, các mức độ khó (Easy, Intermediate, Advanced, Evil) xếp hạng độ khó dựa trên mức độ phức tạp của các phương pháp suy luận logic mà người chơi phải áp dụng. Đề càng dễ, người chơi chỉ cần dùng các quy tắc quét hàng/cột đơn giản. Đề càng khó, người chơi buộc phải nhìn ra các mô hình phức tạp (X-Wing, Swordfish...) để loại trừ ẩn số.

Theo góc nhìn của máy, giải thuật DFS hoàn toàn không có khả năng nhận diện các mô hình logic hay sử dụng bất kỳ thông tin gợi ý nào. Nó duyệt không gian trạng thái một cách tuần tự: luôn bắt đầu từ ô trên cùng bên trái, và tại mỗi ô luôn ưu tiên thử điền các số thứ tự từ 1 đến 9.

Do đó, thời gian thực thi của DFS không phụ thuộc vào "độ khó logic", mà phụ thuộc hoàn toàn vào sự tương thích giữa cấu trúc dữ kiện ban đầu và thứ tự duyệt của thuật toán.

Nếu một đề EVIL có cấu hình các ô trống vô tình thuận lợi cho việc duyệt tuần tự từ trái sang phải, các giá trị đúng nằm ở đầu dải (1, 2, 3), DFS sẽ tiến thẳng đến trạng thái đích rất nhanh (như trường hợp ADVANCED_2). Ngược lại, đối với một đề INTERMEDIATE, do thiếu các hàm đánh giá cục bộ, nếu DFS đoán sai một con số ở những hàng đầu tiên, thuật toán sẽ bị cuốn sâu vào cây tìm kiếm. Nó tiếp tục sinh ra hàng trăm ngàn trạng thái con sai lệch ở các hàng bên dưới, chạm ngõ cụt liên tục và buộc phải thực hiện backtracking để lùi về sửa lại sai lầm.

Kết luận: Kết quả thực nghiệm cho thấy hiệu suất của DFS bị chi phối bởi cách phân bố các con số ban đầu chứ không phải độ khó logic của trò chơi. Vì chỉ duyệt "mù" theo một thứ tự máy móc mà không có khả năng suy luận loại trừ như con người, DFS tỏ ra cực kỳ thiếu ổn định khi gặp các cấu hình Sudoku bất lợi. Điểm yếu này chính là bước đệm để chúng ta tìm đến các phương pháp duyệt Heuristic trong phần tiếp theo.

2.3 Giải thuật Heuristic trong việc giải bài toán Sudoku

2.3.1 Giới thiệu

Phương pháp Heuristic sử dụng khi giải bài toán Sudoku là một kỹ thuật giải quyết bài toán kết hợp giữa trực giác của con người và các chiến lược tối ưu hóa nhằm giải các câu đố Sudoku. Phương pháp này dựa trên cơ chế lan truyền ràng buộc (constraint propagation), trong đó thuật toán loại bỏ các giá trị không thể xảy ra đối với mỗi ô dựa trên các ràng buộc hiện có của bài toán.

Cụ thể, thuật toán duy trì một tập các giá trị ứng viên có thể cho mỗi ô chưa được điền trong bảng. Dựa trên các ràng buộc của Sudoku (hàng, cột và khối 3×3), các giá trị không hợp lệ sẽ dần được loại bỏ khỏi tập ứng viên này.

Ngoài ra, thuật toán có thể áp dụng một số kỹ thuật suy luận thường được con người sử dụng khi giải Sudoku, chẳng hạn như *naked singles* hoặc *hidden pairs*, để từng bước xác định các giá trị chắc chắn cho các ô còn trống.

Trong trường hợp không thể suy ra trực tiếp một giá trị hợp lệ, thuật toán sẽ kết hợp với phương pháp *backtracking* để thử các giá trị khả thi dựa trên các phỏng đoán có định hướng. Nếu phát sinh xung đột với các ràng buộc của Sudoku, thuật toán sẽ quay lui (backtrack) và thử các lựa chọn khác cho đến khi tìm được lời giải hợp lệ.

2.3.2 Môt vài công trình tương tự

Bài toán giải Sudoku đã được nghiên cứu rộng rãi trong lĩnh vực các bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problems - CSPs) [2], đặc biệt với các phương pháp kết hợp giữa backtracking và các heuristic. Những phương pháp này nhằm mục tiêu thu hẹp không gian tìm kiếm của lời giải và cải thiện hiệu quả thời gian giải bài toán.

Simonis [1] đã đề xuất mô hình hóa Sudoku như một bài toán CSP sử dụng ràng buộc *all-different*, đảm bảo rằng mỗi chữ số chỉ xuất hiện đúng một lần trong mỗi hàng, mỗi cột và mỗi khối 3×3. Cách tiếp cận này đặt nền tảng cho các kỹ thuật lan truyền ràng buộc (constraint propagation) như *Arc Consistency* (AC-3) và *Forward Checking*, vốn được sử dụng rộng rãi để loại bỏ sớm các giá trị không phù hợp trong quá trình tìm kiếm.

Norvig [3] đã giới thiệu một bộ giải Sudoku nổi tiếng được cài đặt bằng Python, kết hợp giữa backtracking và lan truyền ràng buộc. Phương pháp này sử dụng heuristic *Minimum Remaining Value* (MRV) để ưu tiên lựa chọn các biến có số lượng giá trị hợp lệ ít nhất, từ đó cải thiện đáng kể hiệu quả của quá trình tìm kiếm.

Rodrigues và cộng sự đã cải thiện hiệu năng của các phương pháp heuristic bằng cách kết hợp đồng thời hai chiến lược *Minimum Remaining Value* (MRV) và *Least Constraining Value* (LCV) [4].

Một số nghiên cứu khác đã thử nghiệm các kỹ thuật lai (hybrid techniques) kết hợp giữa backtracking, lan truyền ràng buộc và các heuristic đặc thù của bài toán nhằm nâng cao hiệu suất giải. Các nghiên cứu này đã chứng minh tính hiệu quả của những phương pháp trên khi áp dụng cho các bài toán phức tạp như Sudoku [5]. Barker và cộng sự [6] đã phát triển một phương pháp kết hợp giữa tìm kiếm cục bộ (local search) và bài toán thỏa mãn ràng buộc, sử dụng backtracking cùng với chiến lược tham lam (greedy) để lựa chọn các biến tiềm năng, từ đó cải thiện đáng kể hiệu quả khi giải các câu đố Sudoku có độ khó cao.

Ngoài các phương pháp dựa trên CSP, việc giải Sudoku cũng được nghiên cứu dưới góc nhìn của các bài toán tối ưu tổ hợp. Sudoku có thể được xem như một trường hợp đặc biệt của bài

toán *Exact Cover*, vốn là một bài toán NP-Complete trong trường hợp tổng quát đối với các lưới Sudoku kích thước $n \times n$ [7]. Thuật toán *Dancing Links* (DLX) của Knuth [8], một kỹ thuật giải bài toán Exact Cover hiệu quả dựa trên thao tác với ma trận thưa, đã được áp dụng thành công để giải Sudoku. DLX hoạt động hiệu quả nhờ khả năng hoàn tác và khôi phục nhanh các quyết định trong quá trình backtracking.

Hướng tiếp cận của nhóm em học hỏi và áp dụng chiến lược tương tự Rodrigues và cộng sự bằng cách triển khai MRV, LCV, Forward Check cùng với các heuristic [9] bổ sung nhằm tối ưu hóa quá trình đưa ra quyết định trong quá trình giải Sudoku.

2.3.3 Hiện thực giải pháp

Bộ giải dựa trên heuristic ưu tiên chọn biến có ít giá trị hợp lệ còn lại nhất (MRV) và sau đó chọn các giá trị ít gây ràng buộc nhất lên các biến khác (LCV), điều này giúp tăng tốc đáng kể quá trình giải bằng cách thu hẹp không gian tìm kiếm và áp dụng thuật toán quay lui (backtracking) khi một giá trị được chọn không chính xác.

- MRV (Minimum Remaining Values): Chọn ô (biến) có ít giá trị khả thi nhất để điền trước
- LCV (Least Constraining Values): Thử giá trị nào ít ảnh hưởng nhất đối với các ô khác trước
- Forward Checking: Kĩ thuật lan truyền ràng buộc (Constraint Propagation). Mỗi khi điền một số vào một ô, ngay lập tức loại bỏ số đó khỏi danh sách các giá trị hợp lệ của các ô cùng hàng, cột, và khối 3x3.

Đầu vào: Ma trận B (board) kích thước 9×9 với một số giá trị được điền sẵn và các số 0 đại diện cho các ô trống.
Đầu ra: Một bảng Sudoku hợp lệ được điền đầy đủ nếu có thể giải được; ngược lại, trả về thông báo "Không có nghiệm".

Algorithm 1 INIT_DOMAINS

```
function INIT_DOMAINS(board)
    domains ← empty dictionary
    for each cell (r, c) in board do
        if board[r][c] = 0 then
            possible ← {1, 2, ..., 9}
            for each neighbor of (r, c) do
                if neighbor has a value then
                    remove that value from possible
                end if
            end for
            domains(r, c) ← possible
        end if
    end for
    return domains
end function
```

Hàm `INIT_DOMAINS` khởi tạo miền giá trị khả dĩ cho các ô trống dựa trên ràng buộc Sudoku.

Algorithm 2 SELECT_MRV

```
function SELECT_MRV(domains)
    return variable in domains with smallest domain size
end function
```

Hàm `SELECT_MRV` chọn ô có ít giá trị khả dĩ nhất (Minimum Remaining Values).

Algorithm 3 LCV_ORDER

```
function LCV_ORDER(var, domains)
    function IMPACT(val)
        count ← 0
        for each neighbor of var do
            if neighbor in domains and val ∈ domains[neighbor] then
                count ← count + 1
            end if
        end for
        return count
    end function
    sort domains[var] by increasing IMPACT(val)
    return sorted list
end function
```

Hàm `LCV_ORDER` sắp xếp giá trị có thể điền theo nguyên tắc (thứ tự) ít gây ảnh hưởng nhất (Least Constraining Value).

Algorithm 4 FORWARD_CHECK and UNDO

```
function FORWARD_CHECK(var, val, domains, removed_list)
    for each neighbor of var do
        if neighbor in domains and val ∈ domains[neighbor] then
            remove val from domains[neighbor]
            append (neighbor, val) to removed_list
            if domains[neighbor] is empty then
                return False
            end if
        end if
    end for
    return True
end function

function UNDO(domains, removed_list)
    for each (neighbor, val) in removed_list do
        add val back to domains[neighbor]
    end for
end function
```

Hai hàm `FORWARD_CHECKING` và `UNDO` thực hiện kiểm tra tiến (forward checking) và hoàn tác (undo). Khi ta định điền một số vào một ô, hàm này sẽ "nhìn về tương lai" bằng cách xóa tạm

thời số đó khỏi domain của các ô hàng xóm. Nếu có ô của hàng xóm nào bị mất hết lựa chọn (domain wipe-out), nó sẽ báo lỗi ngay để thuật toán không đi vào vết xe đổ.

Algorithm 5 SOLVE

```
function SOLVE(board, domains)
    if domains is empty then
        return True
    end if
    var ← SELECT_MRV(domains)
    (r, c) ← coordinates of var
    sorted_values ← LCV_ORDER(var, domains)
    for each val in sorted_values do
        board[r][c] ← val
        removed_list ← empty list
        remove var from domains
        if FORWARD_CHECK(var, val, domains, removed_list) then
            if SOLVE(board, domains) then
                return True
            end if
        end if
        restore var to domains
        UNDO(domains, removed_list)
        board[r][c] ← 0
    end for
    return False
end function
```

Hàm `SOLVE` là thuật toán quay lui kết hợp MRV, LCV và forward checking để giải Sudoku hiệu quả.

2.3.4 Kết quả

Dưới đây là Hình ảnh kết quả chạy chương trình minh họa (có in ra thời gian, bộ nhớ, số nodes) bảng thống kê sau khi chạy chương trình với bộ 8 input.

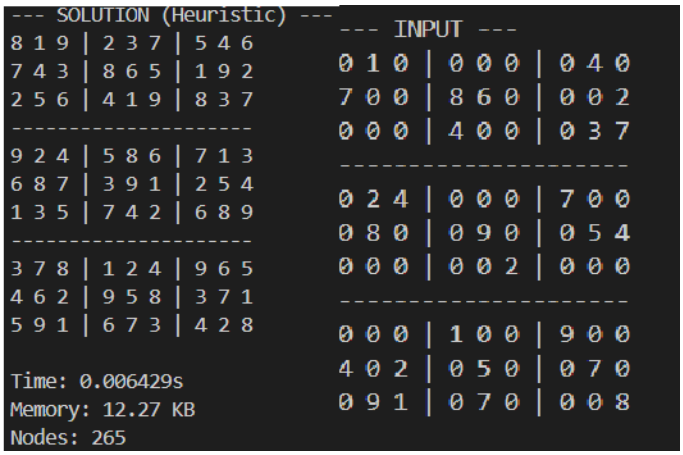

Figure 3: *Hình ảnh kết quả của chương trình chạy Heuristic*

Table 2: Bảng thống kê hiệu suất giải thuật Heuristic đối với các Input

Input	**Time (ms)**	**Memory (KB)**	**Số nodes đã duyệt**	**Kết quả**
EASY_1	1.12	15.16	55	Success
EASY_2	6.97	10.12	495	Success
INTERMEDIATE_1	1.19	6.11	75	Success
INTERMEDIATE_2	4.56	11.83	265	Success
ADVANCED_1	0.98	6.61	51	Success
ADVANCED_2	1.65	6.72	61	Success
EVIL_1	1.70	9.30	95	Success
EVIL_2	1.71	9.04	102	Success

2.4 So sánh giữa Blind Search (DFS) và Heuristic (MRV + LCV)

Để đánh giá hiệu quả của 2 cách tiếp cận, nhóm chúng em đã chạy giải thuật trên bộ 8 input được lấy từ trang (Puzzle Sudoku). Bảng bên dưới tổng kết lại thời gian giải trung bình (với đơn vị ms) đối với mỗi phương pháp.

Các thí nghiệm được thực hiện trên một máy tính cá nhân với cấu hình như sau: bộ vi xử lý Intel® Core™ i5-12450HX (12 luồng xử lý) với xung nhịp cơ bản khoảng 2.4 GHz, cùng với 12 GB RAM. Hệ điều hành sử dụng là Windows 11 Home Single Language 64-bit (Build 26200), hỗ trợ DirectX 12.

Chương trình được cài đặt bằng ngôn ngữ Python. Việc đo thời gian thực thi sử dụng hàm `time.perf_counter` để đảm bảo độ chính xác cao, trong khi bộ nhớ tiêu tốn được ghi nhận thông qua thư viện `tracemalloc`. Tất cả các phép đo được thực hiện trong cùng một môi trường nhằm đảm bảo tính công bằng khi so sánh giữa hai phương pháp DFS và heuristic.

Table 3: Speedup Ratio of Heuristic Solver Over DFS (Per Input)

Input	**DFS (a) (ms)**	**Heuristic (b) (ms)**	**Tỷ lệ (a/b)**	**Tăng tốc**
EASY_1	44.51	1.12	39.74	∼39.7x faster
EASY_2	157.46	6.97	22.59	∼22.6x faster
INTERMEDIATE_1	40.37	1.19	33.95	∼34.0x faster
INTERMEDIATE_2	1055.00	4.56	231.36	∼231.4x faster
ADVANCED_1	204.96	0.98	209.14	∼209.1x faster
ADVANCED_2	55.16	1.65	33.43	∼33.4x faster
EVIL_1	519.01	1.70	305.30	∼305.3x faster
EVIL_2	268.63	1.71	157.09	∼157.1x faster

Kết quả thực nghiệm cho thấy cách tiếp cận sử dụng heuristic (kết hợp MRV, LCV và Forward Checking) vượt trội rõ rệt so với phương pháp tìm kiếm theo chiều sâu (Depth-First Search - DFS) truyền thống. Trên tất cả các bộ dữ liệu thử nghiệm, giải thuật heuristic đều đạt được tốc độ nhanh hơn đáng kể, với hệ số tăng tốc dao động từ khoảng 22× đến hơn 300×.

Sự cải thiện hiệu năng này đến từ nhiều yếu tố quan trọng. Trước hết, heuristic *Minimum Remaining Values (MRV)* giúp lựa chọn biến có miền giá trị nhỏ nhất, từ đó làm giảm hệ số phân nhánh của cây tìm kiếm ngay từ đầu. Điều này cho phép phát hiện sớm các xung đột và tránh việc duyệt các nhánh không khả thi.

Bên cạnh đó, heuristic *Least Constraining Value (LCV)* tối ưu thứ tự thử giá trị bằng cách ưu tiên các giá trị gây ít ràng buộc nhất lên các biến lân cận. Nhờ đó, khả năng duy trì tính nhất quán của các bước gán tiếp theo được tăng lên, đồng thời giảm số lần quay lui (backtracking).

Ngoài ra, kỹ thuật *Forward Checking* đóng vai trò quan trọng trong việc cắt tỉa không gian tìm kiếm bằng cách loại bỏ ngay các giá trị không hợp lệ khỏi miền của các biến liên quan sau mỗi phép gán. Điều này giúp giảm đáng kể số lượng trạng thái cần xét.

Ngược lại, DFS thực hiện tìm kiếm mù với thứ tự biến và giá trị cố định, dẫn đến việc phải duyệt một số lượng lớn các trạng thái không cần thiết. Khi độ khó của bài toán tăng lên (đặc biệt ở các mức ADVANCED và EVIL), nhược điểm này càng trở nên rõ rệt, khiến thời gian chạy tăng nhanh theo cấp số mũ.

Tổng thể, kết quả thực nghiệm khẳng định rằng việc áp dụng heuristic trong bài toán thỏa mãn ràng buộc như Sudoku giúp cải thiện đáng kể hiệu năng và khả năng mở rộng của giải thuật.

3 Trò chơi Chess Ranger

3.1 Mô tả trò chơi

3.1.1 Luật chơi

Trò chơi được ham khảo trên trang web **Chess Ranger**, mọi người có thể truy cập vào để tìm hiểu rõ hơn và chơi thử nếu muốn.

Chess Ranger là một bài toán logic puzzle được xây dựng trên bàn cờ kích thước 8×8. Bài toán sử dụng nhiều loại quân cờ vua khác nhau, cụ thể là Knight (N), Bishop (B), Rook (R), Queen (Q), King (K) và Pawn (P), mỗi loại tuân theo các quy tắc di chuyển đặc trưng riêng.

Mỗi quân cờ tuân theo luật di chuyển và bắt quân giống như trong cờ vua. Luật chơi của bài toán như sau:

- Trong mỗi bước đi, một quân cờ có thể **ăn (capture)** một quân cờ khác nếu nước đi hợp lệ theo luật của quân đó.
- Khi một quân cờ ăn quân cờ khác:
  - Quân bị ăn sẽ bị loại khỏi bàn cờ
  - Quân ăn sẽ di chuyển tới vị trí của quân bị ăn.
- Trò chơi tiếp tục cho đến khi chỉ còn lại một quân cờ trên bàn cờ.

Mục tiêu của bài toán là: Tìm một chuỗi các bước bắt quân hợp lệ sao cho từ trạng thái ban đầu có nhiều quân cờ, cuối cùng chỉ còn lại một quân cờ duy nhất.

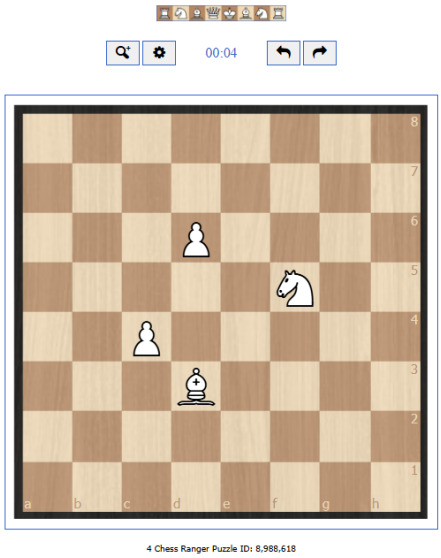

Figure 4: *Hình ảnh trò chơi Chess ranger Puzzle*

3.1.2 Biểu diễn bài toán và không gian trạng thái

Trong chương trình, trạng thái bàn cờ được biểu diễn bằng một `dictionary` trong Python:

```
board = {(row, col): piece_type}
```

Ví dụ:

```
{
 (1,1): 'N',
 (1,6): 'B',
 (2,1): 'P',
 ...
}
```

Trong đó:

- `(row, col)` là vị trí của quân cờ trên bàn cờ.
- `piece_type` là loại quân cờ.

Trạng thái (State)

Một trạng thái được định nghĩa là:

$$S = \text{tập các quân cờ còn lại trên bàn cờ}$$

Trạng thái ban đầu

Trạng thái ban đầu là cấu hình các quân cờ được cung cấp từ *input*.

Trạng thái đích

Trạng thái đích đạt được khi: `len(board)` $== 1$ tức là chỉ còn lại một quân cờ trên bàn cờ.

Toán tử (Operators)

Một toán tử tương ứng với: Một quân cờ A ăn một quân cờ B. Nếu `validcapture(pieceA, posA, posB)` thỏa mãn luật di chuyển của quân cờ

Sau khi thực hiện phép ăn, trạng thái bàn cờ được cập nhật như sau:

```
del board[posA]
del board[posB]
board[posB] = pieceA
```

Không gian trạng thái

Không gian trạng thái bao gồm: tất cả các cấu hình bàn cờ có thể sinh ra từ việc thực hiện các nước đi hợp lệ.

Vì mỗi lần ăn làm giảm số quân cờ đi 1, nên độ sâu tối đa của cây tìm kiếm là: $n - 1$, với n là số quân ban đầu.

3.2 Giải thuật Blind Search

3.2.1 Mô tả giải thuật sử dụng

Giải thuật Tìm kiếm theo chiều sâu (Depth-First Search - DFS) thuộc nhóm giải thuật tìm kiếm mù (Blind Search), nghĩa là quá trình duyệt không gian trạng thái diễn ra một cách hệ thống theo một trình tự cố định mà không sử dụng bất kỳ hàm đánh giá hay thông tin gợi ý nào để ưu tiên các nhánh.

Thuật toán tiến hành duyệt không gian bằng cách liên tục sinh ra các nước ăn quân hợp lệ và đi sâu vào một nhánh. Khi thuật toán rơi vào ngõ cụt (không còn nước ăn quân nào nhưng bàn cờ vẫn còn nhiều hơn 1 quân), cơ chế quay lui (Backtracking) được kích hoạt tuân theo nguyên tắc LIFO (Last In, First Out). Thuật toán sẽ lùi về trạng thái ngay trước đó để tiếp tục thử nhánh ứng viên tiếp theo.

3.2.2 Mô hình hóa bài toán và hiên thực

Trò chơi Chess Ranger áp dụng giải thuật DFS bằng cách sử dụng một cấu trúc dữ liệu Ngăn xếp (Stack) để quản lý các trạng thái chờ duyệt. Mỗi phần tử trong Stack chứa trạng thái hiện tại của bàn cờ và mảng lộ trình các nước đi đã thực hiện.

Đồng thời, để khắc phục điểm yếu dễ bị rơi vào vòng lặp vô hạn của duyệt đồ thị, thuật toán được hiện thực thêm một tập hợp đánh dấu các trạng thái đã thăm. Mỗi khi lấy ra một trạng thái từ Stack, bàn cờ sẽ được chuyển đổi thành một chuỗi Tuple bất biến và lưu vào tập da_tham. Các trạng thái con sinh ra nếu trùng lặp với tập này sẽ bị bỏ qua, giúp tiết kiệm bộ nhớ và chặn các nhánh lặp lại dư thừa.

3.2.3 Kết quả chạy thử

Quá trình thực nghiệm được tiến hành bằng cách dùng thuật toán DFS để giải các đề bài Chess Ranger với số lượng quân cờ từ 4 quân lên đến 11 quân. Các trạng thái kết thúc đều lấp đầy điều kiện chỉ còn 1 quân trên bàn. Dưới đây là bảng thống kê số liệu thu thập được từ các file dữ kiện:

Table 4: Bảng thống kê hiệu suất giải thuật DFS đối với các Input

Input	Giải thuật	Thời gian (s)	Bộ nhớ (KB)	Nodes	Kết quả
bc_4_1	DFS	0.000095	3.34	4	Thành công
bc_4_2	DFS	0.0001095	3.83	7	Thành công
bc_6_1	DFS	0.0001836	9.05	10	Thành công
bc_6_2	DFS	0.0006279	19.81	62	Thành công
bc_8_1	DFS	0.0007681	29.09	64	Thành công
bc_8_2	DFS	0.0003548	18.19	19	Thành công
bc_10_1	DFS	0.0030659	78.43	243	Thành công
bc_10_2	DFS	0.0028021	80.02	232	Thành công
bc_11_1	DFS	0.0277315	437.68	2675	Thành công
bc_11_2	DFS	0.0533161	966.86	4962	Thành công

3.2.4 Phân tích hiệu suất của DFS

Hiệu suất của DFS đối với trò chơi Chess Ranger được phân tích qua hai phương diện: độ phức tạp không gian và độ phức tạp thời gian.

Độ phức tạp không gian (tiêu tốn bộ nhớ): Giải thuật DFS trong trò chơi Chess Ranger tiêu thụ bộ nhớ tăng dần theo độ phức tạp của bài toán. Từ input `bc_4_1` chỉ tốn 3.34 KB bộ nhớ, dung lượng này đã tăng lên tới 966.86 KB ở input `bc_11_2`. Nguyên nhân cốt lõi là do thuật toán phải lưu trữ mảng các trạng thái đã thăm (*visited states*) trong vùng nhớ Heap nhằm tránh việc lặp lại các trạng thái đã duyệt. Khi số lượng nút trên cây tìm kiếm tăng lên, số trạng thái cần lưu trữ cũng tăng theo, khiến lượng bộ nhớ cần sử dụng phình to đáng kể.

Độ phức tạp thời gian: Có thể quan sát thấy sự bất tương xứng rõ rệt giữa cấu trúc bàn cờ và chi phí thời gian cũng như số lượng nút phải duyệt. Khi số lượng quân cờ trên bàn tăng lên (từ 4 quân lên 11 quân), hệ số nhánh (*branching factor*) cho các nước ăn quân tăng mạnh. Thuật toán DFS hoàn toàn không có khả năng nhận diện các mô hình logic hay sử dụng bất kỳ thông tin gợi ý nào. Thay vào đó, nó duyệt không gian trạng thái một cách tuần tự. Vì vậy, đối với các bàn cờ có 11 quân, thuật toán liên tục sinh ra nhiều trạng thái sai lệch, gặp các ngõ cụt và buộc phải thực hiện *backtracking* để quay lui, dẫn đến việc phải duyệt gần 5000 nút.

Một điểm đáng chú ý là giữa các input `bc_6_1` và `bc_6_2`, hay `bc_8_1` và `bc_8_2`, mặc dù có cùng số lượng quân cờ nhưng số nút mở rộng lại chênh lệch đáng kể (lần lượt là 10 và 62, 64 và 19). Điều này cho thấy thuật toán không có khả năng nhận diện các mô hình logic hay tận dụng bất kỳ thông tin gợi ý nào. Nó đơn thuần duyệt không gian trạng thái theo thứ tự được lập trình sẵn; quân cờ nào được xét trước sẽ được mở rộng trước.

Do đó, thời gian và bộ nhớ cần sử dụng không hoàn toàn phụ thuộc vào độ khó logic của bài toán mà còn phụ thuộc vào cách sắp xếp dữ liệu ban đầu. Các bàn cờ `bc_6_1` và `bc_8_2`, với vị trí quân cờ vô tình phù hợp với thứ tự duyệt của thuật toán, cho phép DFS nhanh chóng tìm ra các nước đi hợp lệ mà ít phải thực hiện backtracking. Nhờ đó, bài toán được giải nhanh hơn đáng kể so với các trường hợp còn lại dù có cùng mức độ khó về mặt logic.

Kết luận: Kết quả thực nghiệm cho thấy hiệu suất của DFS bị chi phối mạnh bởi cách phân bố các quân cờ ban đầu hơn là độ khó thực sự của bài toán. Việc duyệt "mù" theo thứ tự cố định khiến DFS thiếu tính ổn định khi gặp các bàn cờ được sắp xếp bất lợi hoặc ngược lại với thứ tự duyệt. Đây chính là động lực để tìm kiếm các phương pháp giải hiệu quả hơn dựa trên *heuristic*, sẽ được trình bày ở phần tiếp theo.

3.3 Giải thuật Heuristic

3.3.1 Mô tả giải thuật sử dụng

Trong bài toán này, nhóm sử dụng thuật toán **A* Search**. A* là một thuật toán tìm kiếm heuristic nổi tiếng dùng để tìm đường đi tối ưu trong không gian trạng thái.

Hàm đánh giá của A* được định nghĩa như sau:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: chi phí từ trạng thái ban đầu đến trạng thái hiện tại.
- $h(n)$: heuristic ước lượng chi phí còn lại để đạt trạng thái đích.

Trong bài toán này:

- $g(n)$: số bước bắt đã thực hiện.
- $h(n)$: ước lượng số bước cần thiết để giảm bàn cờ về còn 1 quân.

Thuật toán sử dụng `priority queue` (thư viện `heapq`) để luôn mở rộng trạng thái có giá trị $f(n)$ nhỏ nhất.

Code cài đặt A* được hiện thực trong hàm: `def astar(initial_board)`

Trong quá trình tìm kiếm, chương trình lưu trữ:

- `open_list`: hàng đợi ưu tiên.
- `visited`: tập các trạng thái đã duyệt.

Mỗi trạng thái được biểu diễn bằng: `frozenset(board.items())` để có thể sử dụng trong cấu trúc `set`.

Nếu tìm được trạng thái chỉ còn 1 quân cờ, thuật toán dừng và trả về lời giải.

3.3.2 Mô hình hóa bài toán và hiên thực

Kiểm tra nước ăn quân hợp lệ

Chương trình định nghĩa các hàm kiểm tra nước đi của từng quân cờ:

```
knight_move()
king_move()
pawn_move()
rook_move()
bishop_move()
queen_move()
```

Sau đó hàm: `valid_capture(piece, a, b, board)` được sử dụng để kiểm tra xem quân `piece` ở vị trí `a` có thể ăn quân tại vị trí `b` hay không.

Sinh trạng thái kế tiếp

Các trạng thái kế tiếp được tạo trong hàm: `generate_successors(board)`. Quá trình sinh trạng thái bao gồm các bước:

1. Duyệt qua mọi cặp quân cờ trên bàn cờ.
2. Kiểm tra khả năng *capture*.
3. Tạo trạng thái bàn cờ mới sau khi ăn quân.
4. Tính giá trị heuristic cho trạng thái mới.

Ngoài ra chương trình còn sử dụng cơ chế **priority ordering** đối với quân bị ăn: `priority = "Q":0, "R":1, "B":2,"N":3, "K":4, "P":5`

Mục đích của cơ chế này là ưu tiên bắt các quân có giá trị cao hơn trước, từ đó giúp giảm số nhánh trong quá trình tìm kiếm và cải thiện hiệu quả của thuật toán.

Hàm heuristic

Hàm heuristic được xây dựng dựa trên hai yếu tố chính nhằm ước lượng mức độ khó của trạng thái bàn cờ.

1. **Số thành phần liên thông của đồ thị capture**

   Đầu tiên, chương trình xây dựng *capture graph* với các quy tắc sau:

   - Mỗi quân cờ được biểu diễn như một đỉnh (node) của đồ thị.
   - Nếu quân A có thể bắt quân B thì tồn tại một cạnh nối giữa hai đỉnh tương ứng.

   Sau khi đồ thị được xây dựng, chương trình tiến hành đếm số *connected components*.

   Ý nghĩa:

   - Nếu bàn cờ có nhiều thành phần liên thông, các quân ở những thành phần khác nhau không thể bắt nhau trực tiếp, do đó cần nhiều bước trung gian để đưa chúng vào cùng một khu vực tương tác.

   Chi phí heuristic của yếu tố này được tính bằng:

$$\text{cost}_1 = \text{components} \times 5$$

2. **Khoảng cách Manhattan trung bình**

   Khoảng cách Manhattan giữa hai quân cờ được tính theo công thức:

$$|x_1 - x_2| + |y_1 - y_2|$$

   Chương trình tính khoảng cách Manhattan cho mọi cặp quân cờ và lấy giá trị trung bình nhằm ước lượng mức độ phân tán của các quân trên bàn cờ.

   Chi phí heuristic tương ứng:

$$\text{cost}_2 = \text{dist} \times 3$$

Công thức heuristic cuối cùng:

$$h = 5 \times \text{components} + 3 \times \text{dist}$$

Sau khi tính toán, giá trị heuristic được chuẩn hóa:

$$h = \max(h, 0)$$

Để tăng hiệu suất, giá trị heuristic của mỗi trạng thái được lưu trữ trong bộ nhớ đệm `heur_cache` nhằm tránh việc tính toán lặp lại nhiều lần cho cùng một trạng thái.

3.3.3 Kết quả thực hiện

Chương trình tiến hành đo các thông số sau:

- số node được mở rộng trong quá trình tìm kiếm,
- thời gian chạy của thuật toán,
- lượng bộ nhớ sử dụng.

Các giá trị này được đo bằng các công cụ trong Python như `time.time()` để tính thời gian thực thi và `tracemalloc` để theo dõi mức sử dụng bộ nhớ.

Ví dụ kết quả thu được từ chương trình:

```
Expanded nodes: ...
Time: ... seconds
Memory: ... KB
```

Ngoài ra, chương trình còn hiển thị lời giải theo từng bước (*step-by-step solution*). Ví dụ:

```
Step 1 : N (1,1) captures piece at (2,3)
Step 2 : ...
```

Sau mỗi bước di chuyển, trạng thái bàn cờ cũng được in ra để thể hiện sự thay đổi của các quân cờ. Điều này giúp trực quan hóa toàn bộ quá trình tìm kiếm và giải bài toán.

3.3.4 Phân tích hiệu suất

Ưu điểm của thuật toán Heuristic: Hàm heuristic được thiết kế dựa trên các đặc điểm quan trọng của bài toán:

1. **Connectivity của capture graph**

   Nếu các quân cờ không nằm trong cùng một vùng capture thì cần nhiều bước trung gian để đưa chúng vào cùng một khu vực tương tác.

2. **Khoảng cách giữa các quân cờ**

   Các quân cờ càng nằm xa nhau thì càng khó tạo ra cơ hội bắt quân.

Nhờ kết hợp hai yếu tố này, heuristic giúp:

- giảm đáng kể số lượng node cần mở rộng,

- hướng quá trình tìm kiếm tới các trạng thái có khả năng giải nhanh hơn.

Độ phức tạp: Trong trường hợp xấu nhất:

- Hệ số nhánh (*branching factor*) có thể rất lớn,
- Không gian trạng thái tăng theo cấp số nhân.

Tuy nhiên, việc sử dụng heuristic giúp:

- Giảm đáng kể kích thước của cây tìm kiếm,
- Tránh việc duyệt nhiều trạng thái không cần thiết.

Hạn chế: Heuristic hiện tại vẫn tồn tại một số hạn chế:

- Không xét chính xác số bước cần thiết để tạo ra chuỗi bắt quân (*capture chain*),
- Khoảng cách Manhattan chỉ là một phép ước lượng tương đối thô,
- Có thể đánh giá sai trong một số cấu hình bàn cờ đặc biệt.

Tuy vậy, heuristic vẫn cho kết quả tốt trong phần lớn các trường hợp thử nghiệm.

3.4 So sánh DFS và Heuristic (A*)

3.4.1 Bảng kết quả thực nghiệm

Bài toán 1

Thuật toán	Expanded Nodes	Time (s)	Memory (KB)
DFS	2675	0.0277	437.68
Heuristic (A*)	686	0.3486	3561.21

Bài toán 2

Thuật toán	Expanded Nodes	Time (s)	Memory (KB)
DFS	4962	0.0533	966.86
Heuristic (A*)	128	0.1313	1302.39

3.4.2 Nhận xét kết quả

1. **Số node mở rộng** Thuật toán Heuristic (A*) mở rộng ít node hơn rất nhiều so với DFS.

   Bài toán 1

   - DFS: 2675 nodes
   - A*: 686 nodes

   ⇒ Số node được giảm khoảng 74%.

   Bài toán 2

- DFS: 4962 nodes
- A*: 128 nodes

$\Rightarrow$ Số node được giảm khoảng 97.4%.

Điều này cho thấy heuristic giúp định hướng tìm kiếm tốt hơn, tránh duyệt nhiều trạng thái không cần thiết. Nguyên nhân là vì heuristic đánh giá trạng thái dựa trên:

- Số *connected components* của capture graph,
- Khoảng cách giữa các quân cờ.

Nhờ đó thuật toán ưu tiên mở rộng những trạng thái có khả năng dẫn tới lời giải nhanh hơn.

2. Thời gian chạy Kết quả thời gian chạy được thể hiện trong bảng sau:

Bài toán	DFS (s)	Heuristic (s)
1	0.0277	0.3486
2	0.0533	0.1313

Trong bài toán 1, DFS chạy nhanh hơn mặc dù mở rộng nhiều node hơn. Nguyên nhân là do heuristic phải thực hiện nhiều phép tính bổ sung cho mỗi trạng thái, bao gồm:

- Tính giá trị heuristic,
- Xây dựng capture graph,
- Tính khoảng cách Manhattan,
- Sử dụng cấu trúc dữ liệu *priority queue*.

Những phép tính này làm tăng chi phí tính toán cho mỗi node. Tuy nhiên, với các bài toán lớn hơn, việc giảm số node mở rộng sẽ giúp A* nhanh hơn đáng kể so với DFS.

Bài toán	DFS (KB)	Heuristic (KB)
1	437	3561
2	966	1302

3. Bộ nhớ sử dụng Heuristic sử dụng nhiều bộ nhớ hơn do cần lưu trữ:

- *Open list* (priority queue),
- Các trạng thái đã thăm (*visited states*),
- Bộ nhớ đệm heuristic (*heuristic cache*).

Trong khi đó DFS chỉ cần lưu trữ:

- Ngăn xếp (*stack*),
- Các trạng thái đã thăm.

Vì vậy DFS thường tiết kiệm bộ nhớ hơn.

4. Chất lượng tìm kiếm Một điểm quan trọng là:

- DFS tìm lời giải theo thứ tự duyệt,
- A* tìm lời giải dựa trên giá trị heuristic tốt nhất.

Do đó A* thường tìm được lời giải ổn định và hiệu quả hơn trong các bài toán có không gian trạng thái lớn.

5. Kết luận Từ các kết quả thực nghiệm có thể rút ra rằng:

- DFS đơn giản và có chi phí tính toán mỗi bước thấp.
- Tuy nhiên DFS mở rộng rất nhiều node và không có định hướng tìm kiếm.

Ngược lại:

- Heuristic (A*) có chi phí tính toán cao hơn cho mỗi node,
- Nhưng giúp giảm đáng kể số trạng thái cần duyệt.

Vì vậy, đối với các bài toán có không gian trạng thái lớn, heuristic search là phương pháp hiệu quả hơn so với các thuật toán tìm kiếm mù (*blind search*).

Phụ lục

Nguồn tham khảo

[1] H. Simonis, "Sudoku as a constraint problem," in *CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems*, vol. 12, Sitges, Spain, Oct. 2005, pp. 13–27.

[2] B. Crawford, M. Aranda, C. Castro, and E. Monfroy, "Using Constraint Programming to Solve Sudoku Puzzles," in *Proceedings of the Third International Conference on Convergence and Hybrid Information Technology (ICCIT)*, Busan, South Korea, 2008, pp. 926–931. doi: 10.1109/ICCIT.2008.154.

[3] P. Norvig, "Solving Every Sudoku Puzzle," 2006. [Online]. Available: https://norvig.com/sudoku.html

[4] C. Rodrigues, E. Galvao, and R. L. Azevedo, "LSVF: A heuristic search to reduce the backtracking calls when solving Constraint Satisfaction Problems," *SI nforme*, vol. 19.

[5] V. Kumar, "Algorithms for Constraint-Satisfaction Problems: A Survey," *AI Magazine*, vol. 13, no. 1, pp. 32–44, 1992.

[6] C. Barker, J. Bridge, and P. Nightingale, "Combining Heuristics, Local Search and Backtracking for Sudoku," in *UK Workshop on Computational Intelligence*, 2013.

[7] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E86-A, no. 5, pp. 1052–1060, 2003.

[8] D. E. Knuth, "Dancing Links," *Journal of Algorithms*, vol. 45, no. 2, pp. 123–145, 2000.

[9] A. Bhattarai, D. Uprety, P. Pathak, S. N. Shrestha, S. Narkarmi, and S. Sigdel, "A Study of Sudoku Solving Algorithms: Backtracking and Heuristic," Department of Computer Science, Kathmandu University, Dhulikhel, Nepal, 2025.